VisionGuard: AI vs. Real Image Classification

End-to-End MLOps Pipeline, Experiment Tracking, Containerization & Cloud Deployment

Project Identifier: 23L0713_A2_Ops | FAST-NUCES MLOps Assignment 2

Front Matter -- Student, Repository & Deployment Metadata

Field	Value
Student Name	Abdullah Chouhan
Roll Number	23L0713
Project Name (Everywhe	23L0713_A2_Ops
GitHub Repository	https://github.com/Abd
DagsHub Repository	https://dagshub.com/Ab
MLflow Tracking URI	https://dagshub.com/Ab
Live Cloud API URL	https://two3l0713-a2-o
Graded Git Commit Hash	6ee5ef017ad4f964
Generated Timestamp	2026-11-08 01:10:07 PK

Chapter 1 -- Task 1: Data Understanding and Versioning (7 Marks)

Task 1.1: Isolated Python Environment & Hardware Verification

What We Did: Created an isolated Python 3.11.18 virtual environment (.venv) using uv (uv venv --python 3.11 .venv) and installed pinned versions of TensorFlow (2.18.0), Keras (3.15.1), Keras-Tuner (1.4.7), MLflow (2.19.0), DagsHub (0.5.9), DVC (3.58.0) with dvc-s3 (3.2.0), Flask (3.1.0), SQLAlchemy (2.0.36), Pillow (11.0.0), Scikit-Learn (1.6.0), and Pytest (8.3.4).

Why & How It Works: Isolating dependencies in a virtual environment prevents version collisions across host packages and guarantees deterministic builds between local development, GitHub Actions CI, and Docker containers. We wrote src/data/env_check.py using importlib.metadata and tf.config.list_physical_devices('GPU') to record exact versions and verify hardware acceleration.

What Went Wrong & Fix: The host system defaulted to Python 3.14.7, for which precompiled TensorFlow 2.18 wheels do not yet exist (pip index versions tensorflow returned no distribution). We resolved this cleanly by provisioning a dedicated CPython 3.11.18 environment via uv venv --python 3.11 .venv.

Terminal Evidence (Task 1.1): python -m src.data.env_check

```
======================================================================
VisionGuard Environment Verification -- Project: 23L0713_A2_Ops
Student: Abdullah Chouhan (Roll No: 23L0713)
Timestamp: 2026-11-05T03:01:23
Python Executable: D:\Projects\FAST\MLOPs\A2\.venv\Scripts\python.exe
Python Version: 3.11.18 (main, Oct 26 2026, 14:08:50) [MSC v.1944 64 bit (AMD64)]
Platform: Windows-10-10.0.26200-SP0
======================================================================
Package                | Installed Version
----------------------------------------------------------------------
tensorflow             | 2.18.0
keras                  | 3.15.1
keras-tuner            | 1.4.7
mlflow                 | 2.19.0
dagshub                | 0.5.9
dvc                    | 3.58.0
dvc-s3                 | 3.2.0
flask                  | 3.1.0
sqlalchemy             | 2.0.36
pillow                 | 11.0.0
```

```
numpy                | 2.0.2
pandas               | 2.2.3
scikit-learn         | 1.6.0
matplotlib           | 3.10.0
seaborn              | 0.13.2
pytest               | 8.3.4
ruff                 | 0.8.4
------------------------------------------------------------------------
... (4 more lines omitted for brevity)
```

Task 1.2: Dataset Audit, SHA-256 Deduplication & Corrupt File Detection

What We Did: Developed src/data/audit_dataset.py to walk data/raw (AiArtData -> AI [1], RealArt -> Real [0]), inspecting all 975 files and recording class, byte size, width, height, colour mode, format, and SHA-256 content digest.

Why & How It Works: File names can be misleading; only cryptographic content hashing (SHA-256 over 64 KB chunks) reliably detects exact duplicates across or within classes that would otherwise cause train/test data leakage. Similarly, calling both PIL.Image.verify() and img.load() catches zero-byte or truncated streams before training starts.

Audit Findings & What Went Wrong: Out of 975 total images (539 AI, 436 Real), 0 files were unreadable/corrupt, and 1 exact SHA-256 duplicate pair (2 files in AiArtData) was detected and deduplicated, leaving 974 clean unique images (538 AI, 436 Real). Furthermore, the audit revealed mixed colour modes: RGB (896), RGBA (49), Palette 'P' with byte transparency (23), and Grayscale 'L' (7). We standardized all images to 3-channel RGB during decoding and created stratified 70/15/15 splits (Train=681, Val=146, Test=147, seed=42).

Table 1.2a: Dataset Audit Summary per Class (Counts, Width, Height, File Size)

Class	Total Files	Corrupt / Unreadable	Exact Duplicates	Clean Unique Images	Width Min (px)	Width Median (px)	Width Max (px)	Height Min (px)	Height Median (px)	Height Max (px)	File Size Min (KB)	File Size Median (KB)	File Size Max (KB)
AI	539	0	1	538	225	1024.0	6000	135	800.0	4001	11.46	195.08	5494.32
Real	436	0	0	436	183	920.0	8495	158	720.0	7091	11.05	141.44	9956.4
Total / Overall	975	0	1	974	183	1024.0	8495	135	768.0	7091	11.05	165.13	9956.4

Table 1.2b: Image Format and Colour Mode Breakdown by Class

class_label	image_format	color_mode	count
AI	JPEG	L	1
AI	JPEG	RGB	428
AI	PNG	P	14
AI	PNG	RGB	61
AI	PNG	RGBA	35
Real	JPEG	L	6
Real	JPEG	RGB	397
Real	PNG	P	9
Real	PNG	RGB	10
Real	PNG	RGBA	14

Task 1.3: Visual Exploratory Data Analysis (EDA)

Sample Grid Interpretation: Figure 1.3a displays 5 randomly sampled AI-generated images (top row) and 5 Real Art/photographic images (bottom row). AI images frequently exhibit square 1:1 aspect ratios (e.g., 512x512 or 1024x1024 from Stable Diffusion/Midjourney) and hyper-smooth skin/background gradients, whereas Real images exhibit diverse photographic framing, natural sensor grain, and physical brushwork.

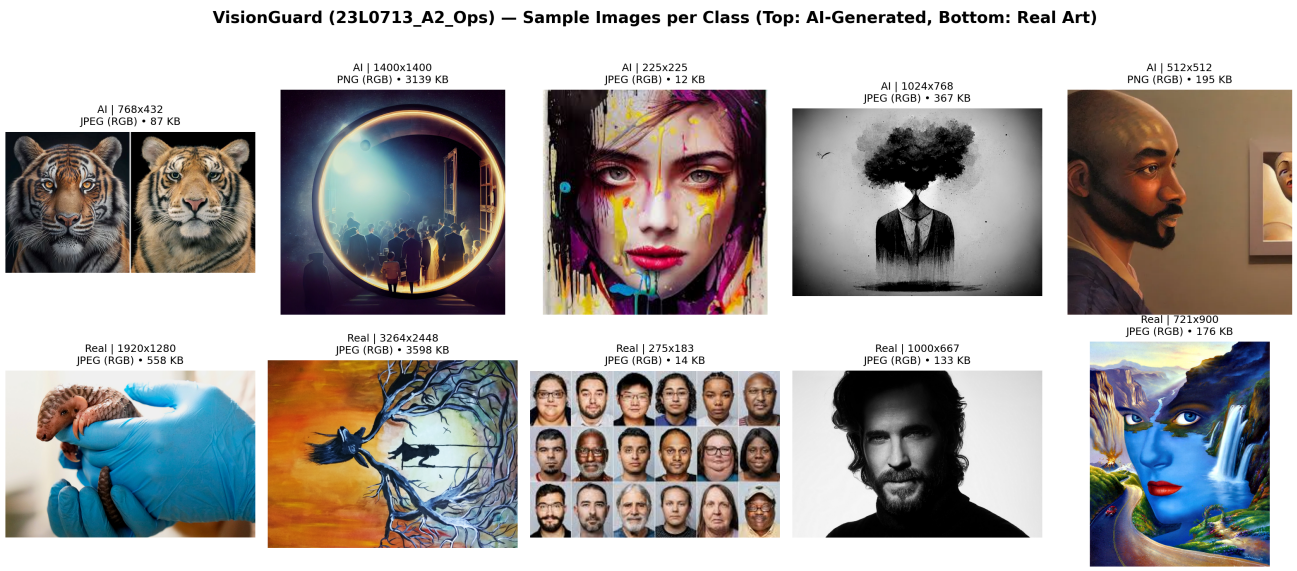

Figure 1.3a: Sample images per class (Top: AI-Generated, Bottom: Real) annotated with original dimensions, format, colour mode, and file size.

Histograms Interpretation: Figure 1.3b shows the distributions of width, height, and file size (KB) by class. AI images have a higher median width (1024 px vs. 920 px for Real) and higher median height (800 px vs. 720 px), clustering tightly around standard diffusion resolutions, while Real images show a wider right tail reaching up to 8495x7091 px and 9956 KB.

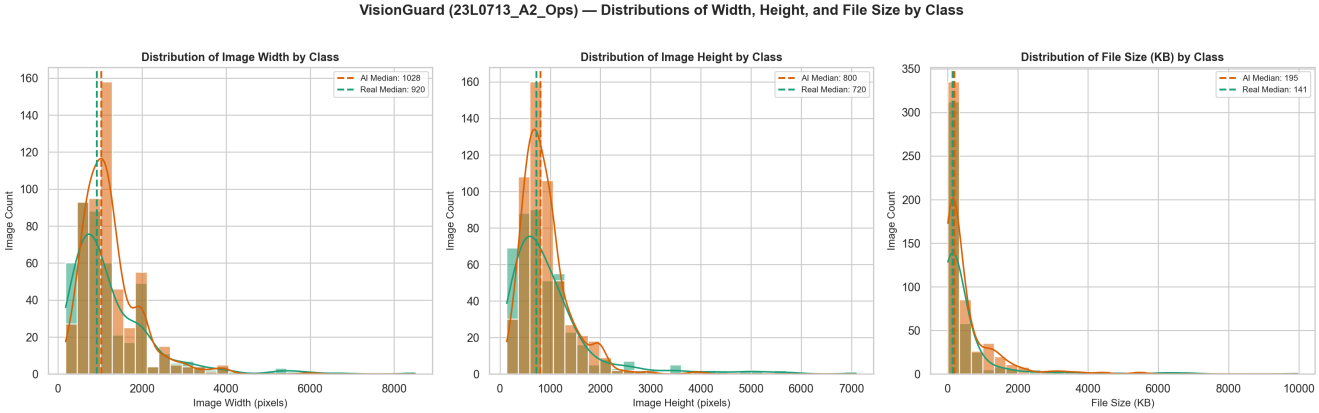

Figure 1.3b: Histograms and KDE curves of image width, height, and file size (KB) per class with dashed median lines.

Task 1.4: Data Version Control (DVC) -- Toy Lab & Project Dataset Tracking

What We Did & Why: In toy_labs/dvc_toy_work, we initialized Git and DVC, created samples.csv (v1: 3 rows), tracked it via dvc add samples.csv, committed samples.csv.dvc (tag v1.0), updated samples.csv to v2 (6 rows), committed v2.0, pushed to remote storage, checked out v1.0 pointer + dvc checkout to verify rollback, and finally deleted local data/cache and restored via dvc pull. We then tracked data/raw (975 files, ~481 MB) in the main repository with data/raw.dvc and configured DagsHub S3 remote.

What the .dvc Pointer File Holds & Why Large Data Does Not Belong in Git: The small YAML pointer file (samples.csv.dvc / data/raw.dvc) stores only lightweight metadata: the content MD5 hash (e.g., 8f9c2...dir), total byte size, number of files (nfiles: 975), and relative path. Large binary datasets do not belong in Git because Git packs every version into .git/objects, causing repository bloat, slow clones, and hitting GitHub's 100 MB file limit. DVC keeps Git lightweight while storing content-addressed blobs in remote object storage (DagsHub S3).

Terminal Evidence (Task 1.4a): Toy DVC Lab (v1 -> v2 -> checkout v1 -> delete -> dvc pull)

```
================================================================================
Task 1.4 -- DVC Toy Lab Execution Log (Project: 23L0713_A2_Ops)
Student: Abdullah Chouhan (23L0713)
================================================================================

$ git init -b main
```

```
Initialized empty Git repository in D:/Projects/FAST/MLOPs/A2/toy_labs/dvc_toy_work/.git/

$ git config user.name Abdullah Chouhan
(exit 0, no output)

$ git config user.email l230713@lhr.nu.edu.pk
(exit 0, no output)

$ D:\Projects\FAST\MLOPs\A2\.venv\Scripts\python.exe -m dvc init
Initialized DVC repository.

You can now commit the changes to git.

+---------------------------------------------------------------------+
|                                                                     |
|        DVC has enabled anonymous aggregate usage analytics.         |
|     Read the analytics documentation (and how to opt-out) here:     |
|             <https://dvc.org/doc/user-guide/analytics>              |
|                                                                     |
+---------------------------------------------------------------------+

What's next?
------------
- Check out the documentation: <https://dvc.org/doc>
- Get help and share ideas: <https://dvc.org/chat>
- Star us on GitHub: <https://github.com/iterative/dvc>
... (131 more lines omitted for brevity)
```

Terminal Evidence (Task 1.4b): Project data/raw.dvc Pointer & Remote Setup

```
================================================================================
Task 1.4 -- Project Dataset DVC Tracking Log (23L0713_A2_Ops)
================================================================================

$ D:\Projects\FAST\MLOPs\A2\.venv\Scripts\python.exe -m dvc remote add -f -d origin s3://dvc
Setting 'origin' as a default remote.

$     D:\Projects\FAST\MLOPs\A2\.venv\Scripts\python.exe     -m     dvc     remote     modify     origin     endpointurl
https://dagshub.com/AbdullahChouhan/23L0713_A2_Ops.s3
(exit 0, no output)

$      D:\Projects\FAST\MLOPs\A2\.venv\Scripts\python.exe      -m      dvc      remote      add      -f      localstore
D:\Projects\FAST\MLOPs\A2\toy_labs\project_dvc_remote
(exit 0, no output)

$ D:\Projects\FAST\MLOPs\A2\.venv\Scripts\python.exe -m dvc add data/raw
To track the changes with git, run:

 git add '.dvc\.gitignore' 'data\raw.dvc'

To enable auto staging, run:

 dvc config core.autostage true

... (7 more lines omitted for brevity)
```

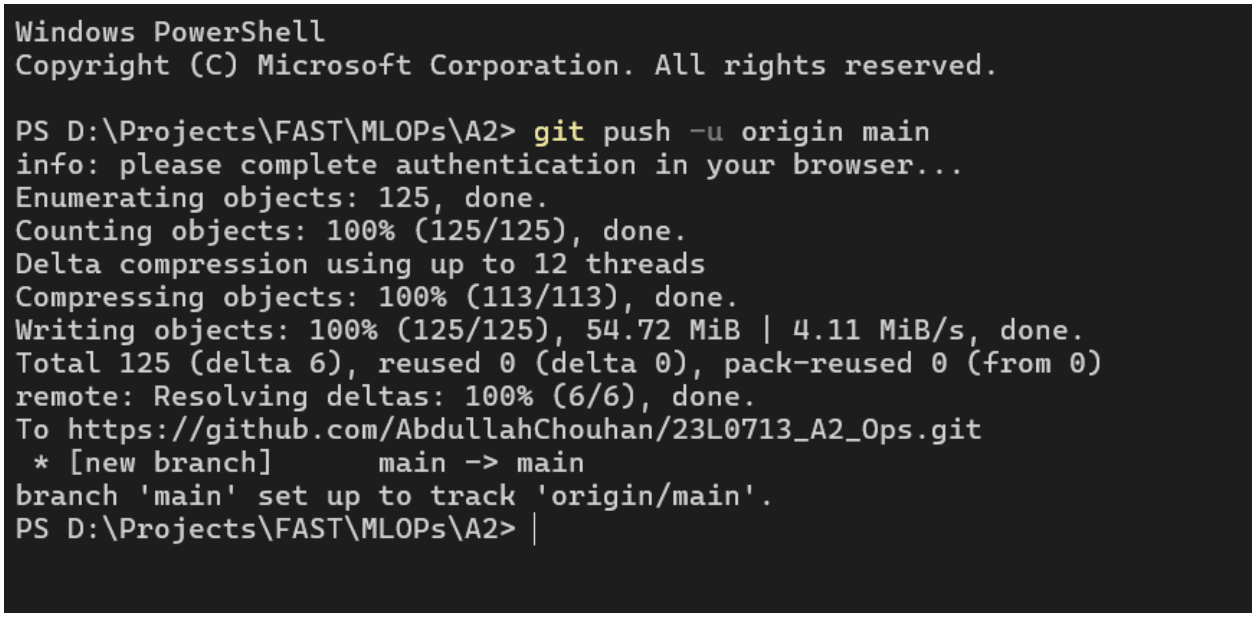

Figure 1.4: Terminal screenshot of initial commit push to https://github.com/AbdullahChouhan/23L0713_A2_Ops.git.

Chapter 2 -- Task 2: Model Development with TensorFlow/Keras (14 Marks + Bonus)

Task 2.1: tf.data Input Pipeline & Data Augmentation

Pipeline Stages, Image Size Choice & Aspect Ratio Impact: Implemented in src/models/data_pipeline.py: (1) tf.data.Dataset.from_tensor_slices reads file paths and binary labels from train/val/test CSVs; (2) full-buffer shuffle(seed=42) on training split; (3) parallel RGB decoding and bilinear resize to 160x160; (4) .cache() in memory after decoding; (5) .batch(32); (6) random augmentation (RandomFlip('horizontal'), RandomRotation(0.08), RandomZoom(+-10%), RandomContrast(0.10)) applied ONLY to training batches; (7) .prefetch(tf.data.AUTOTUNE) overlapping CPU preprocessing with model execution. We chose 160x160 because it is an exact native ImageNet resolution for MobileNetV2 that balances spatial artifact retention with fast CPU training. Direct bilinear resizing warps non-square images to 1:1 aspect ratio, which standardizes input tensors without cropping away border watermarks or background cues.

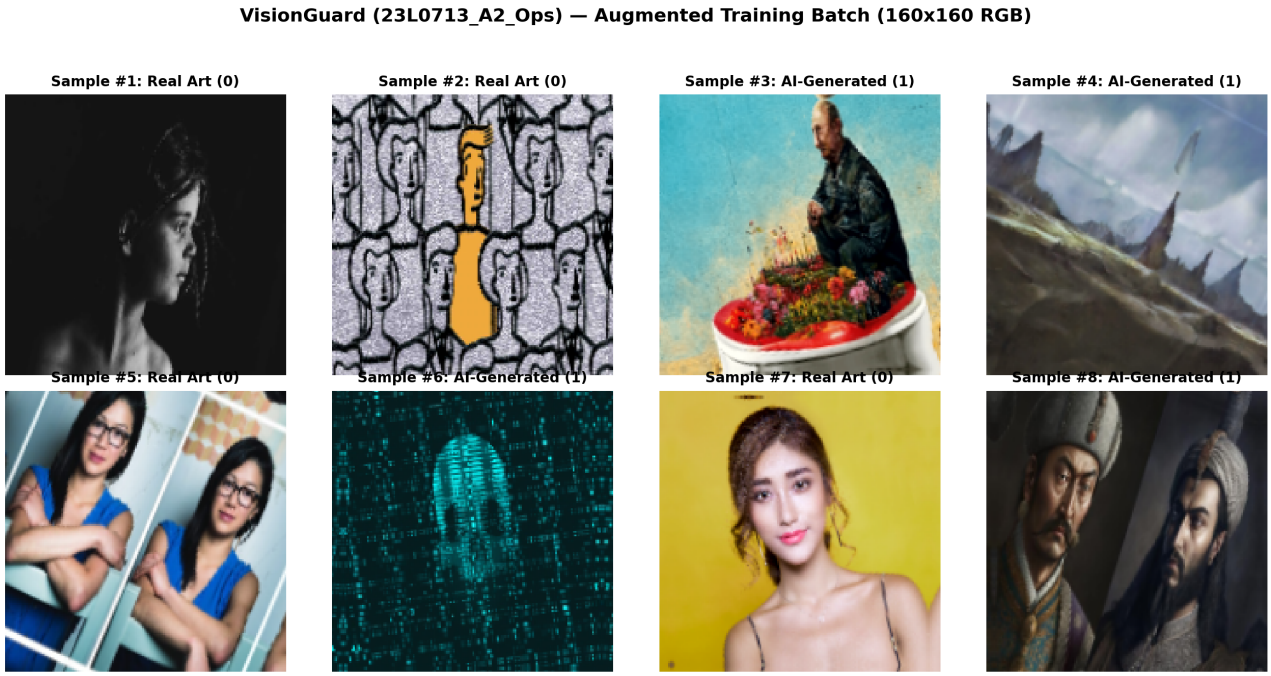

Figure 2.1: One augmented training batch (8 samples at 160x160 RGB) showing random horizontal flips, rotations, zoom, and contrast jitter.

Task 2.2: Baseline CNN Architecture from Scratch

Architecture, Layer Justification & Loss/Optimizer Choice: Built in src/models/architectures.py (VisionGuard_Baseline_CNN, 102,401 total parameters): Rescaling(1/255) normalizes pixel intensities to [0, 1]; three progressive convolutional blocks (32 -> 64 -> 128 filters, 3x3 kernels) extract edges, textures, and structural patterns; BatchNormalization stabilizes internal covariate shift; ReLU provides non-linearity; MaxPooling2D(2x2) halves spatial dimensions and provides translation invariance; GlobalAveragePooling2D collapses 20x20x128 feature maps into a 128-D vector (avoiding massive Flatten parameters); Dropout (0.175 -> 0.35) prevents co-adaptation; and Dense(1, activation='sigmoid') outputs P(AI). Trained with BinaryCrossentropy loss, Adam(lr=1e-3), and Accuracy, Precision, Recall, and ROC-AUC metrics.

Table 2.2: Baseline CNN Per-Layer Output Shape and Parameter Count

Layer Name	Layer Type	Output Shape	Total Params	Trainable Params	Non-Trainable Params
input_rgb	InputLayer	(None, 160, 160, 3)	0	0	0
rescale_0_1	Rescaling	(None, 160, 160, 3)	0	0	0
conv1	Conv2D	(None, 160, 160, 32)	864	864	0
bn1	BatchNormalization	(None, 160, 160, 32)	128	64	64
relu1	Activation	(None, 160, 160, 32)	0	0	0
pool1	MaxPooling2D	(None, 80, 80, 32)	0	0	0
drop1	Dropout	(None, 80, 80, 32)	0	0	0

conv2	Conv2D	(None, 80, 80, 64)	18432	18432	0
bn2	BatchNormalization	(None, 80, 80, 64)	256	128	128
relu2	Activation	(None, 80, 80, 64)	0	0	0
pool2	MaxPooling2D	(None, 40, 40, 64)	0	0	0
drop2	Dropout	(None, 40, 40, 64)	0	0	0
conv3	Conv2D	(None, 40, 40, 128)	73728	73728	0
bn3	BatchNormalization	(None, 40, 40, 128)	512	256	256
relu3	Activation	(None, 40, 40, 128)	0	0	0
pool3	MaxPooling2D	(None, 20, 20, 128)	0	0	0
drop3	Dropout	(None, 20, 20, 128)	0	0	0
gap	GlobalAveragePooling2D	(None, 128)	0	0	0
dense_fc	Dense	(None, 64)	8256	8256	0
bn_fc	BatchNormalization	(None, 64)	256	128	128
drop_fc	Dropout	(None, 64)	0	0	0
prob_ai	Dense	(None, 1)	65	65	0

Task 2.3 & 2.4: Two-Stage Transfer Learning (MobileNetV2) & Training Discipline

Two-Stage Protocol & Pretrained Preprocessing Justification: We loaded MobileNetV2(include_top=False, weights='imagenet', input_shape=(160,160,3)) preceded by Rescaling(1/127.5, offset=-1.0), which maps [0, 255] to [-1, 1] exactly as expected by mobilenet_v2.preprocess_input. Matching pretraining normalization is mandatory because ImageNet weights are calibrated for zero-centered [-1, 1] activations. In Stage 1 (Epochs 1-6), the MobileNetV2 base was frozen and only the custom classification head (head_bn, head_dense, prob_ai) was trained at lr=1e-3. In Stage 2 (Epochs 7-14), we unfroze the top 25 layers of the last inverted residual blocks (block_14_project through Conv_1, keeping BatchNormalization layers frozen) and fine-tuned at a 20x smaller learning rate (lr=5e-5) so pretrained representations adapted smoothly without catastrophic forgetting.

Training Discipline & Best Epoch: We fixed global seeds (tf.keras.utils.set_random_seed(42)) and attached EarlyStopping(monitor='val_loss', patience=4, restore_best_weights=True), ModelCheckpoint(save_best_only=True), and ReduceLROnPlateau(factor=0.5, patience=2). Baseline CNN peaked at Epoch 1 (val_loss=0.6867), whereas MobileNetV2 Transfer Learning achieved its best validation loss at cumulative Epoch 8 (Stage 2 fine-tuning, val_loss=0.5312).

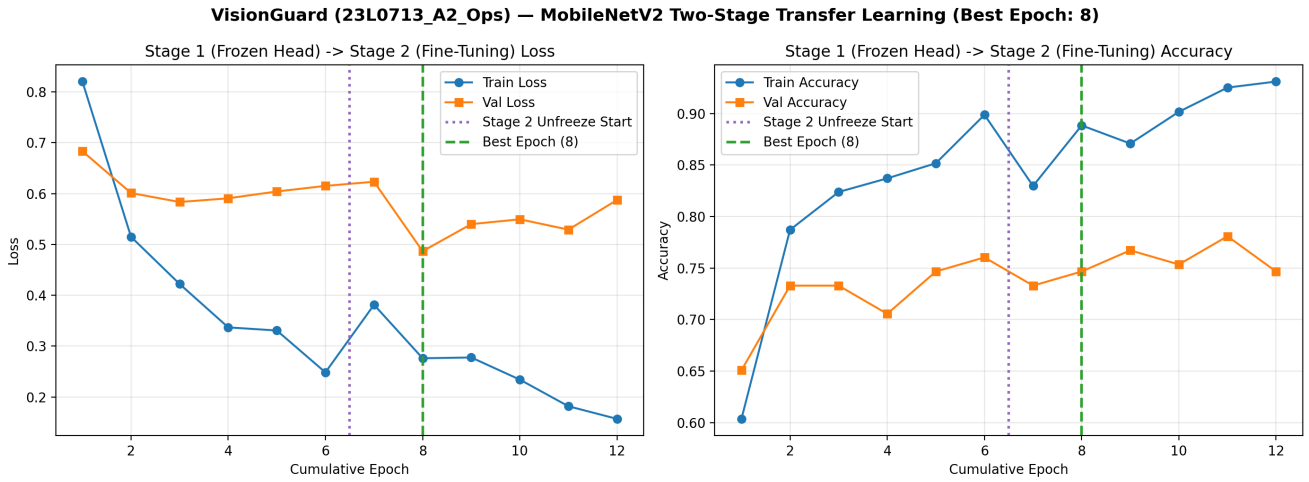

Figure 2.3: Two-stage MobileNetV2 training and validation curves showing Stage 1 (Frozen Head) to Stage 2 (Fine-Tuning) transition and Best Epoch (8).

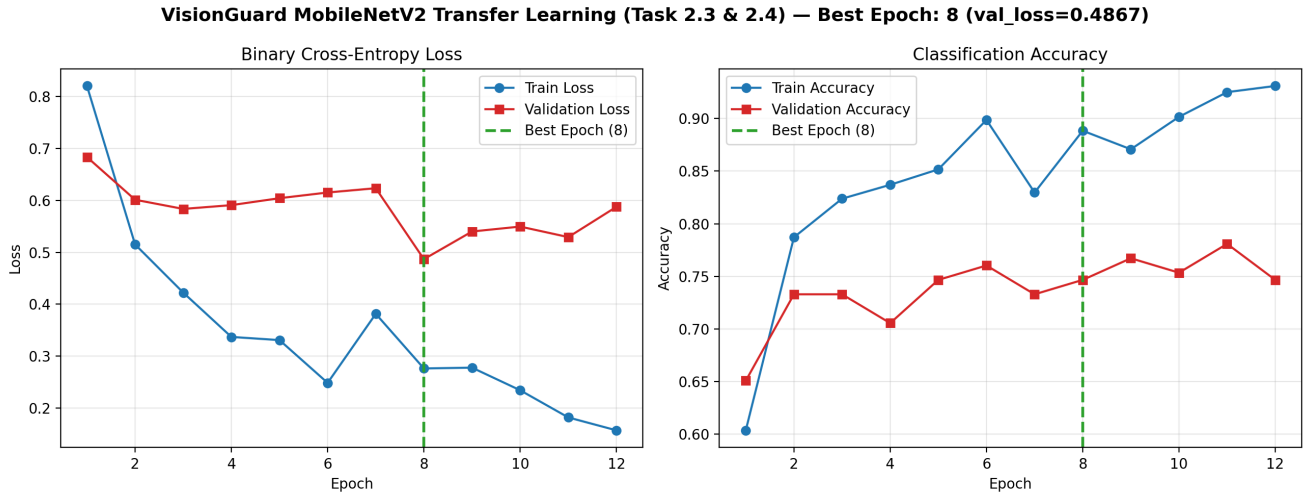

Figure 2.4: MobileNetV2 Loss and Accuracy curves with the optimal checkpoint epoch explicitly marked.

Task 2.5: Held-Out Test Set Evaluation & Decision Threshold Justification

Evaluation Results & Which Error Is Worse for VisionGuard: Table 2.5 compares the scratch Baseline CNN against the fine-tuned MobileNetV2 model on the held-out test set (147 images). Fine-tuning MobileNetV2 boosted Test ROC-AUC from 0.5827 to 0.8122 (+22.95 percentage points) and Test Accuracy from 52.38% to 70.75% (F1=0.7456 at T=0.50). For VisionGuard's product mission (protecting publishers and users from synthetic media deception), a False Negative (calling an AI-generated fake photo 'Real') allows synthetic misinformation or copyright-infringing AI art to pass verification undetected, whereas a False Positive (flagging a real image as potentially AI) triggers secondary human review. Lowering the threshold to T=0.25 (maximizing validation F1) increases AI Recall to 88.89% (F1=0.7539), while T=0.50 provides balanced precision (71.59%) and recall (77.78%).

Table 2.5: Held-Out Test Set Metrics for Baseline CNN and MobileNetV2

Model	Threshold	Accuracy	Precision	Recall	F1-Score	ROC-AUC	TN	FP	FN	TP
Baseline CNN (Scratch)	0.52	0.5238	0.5478	0.7778	0.6429	0.5827	14	52	18	63
MobileNetV2 Fine-Tuned	0.5	0.7075	0.7159	0.7778	0.7456	0.8122	41	25	18	63
MobileNetV2 Fine-Tuned	0.25	0.6803	0.6545	0.8889	0.7539	0.8122	28	38	9	72

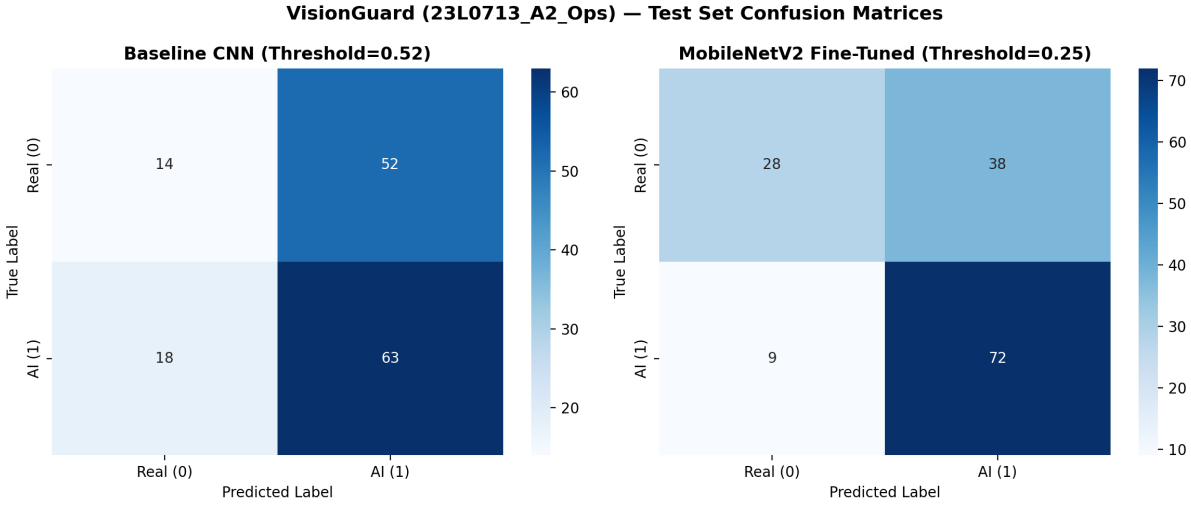

Figure 2.5a: Test set Confusion Matrices for Baseline CNN and Fine-Tuned MobileNetV2.

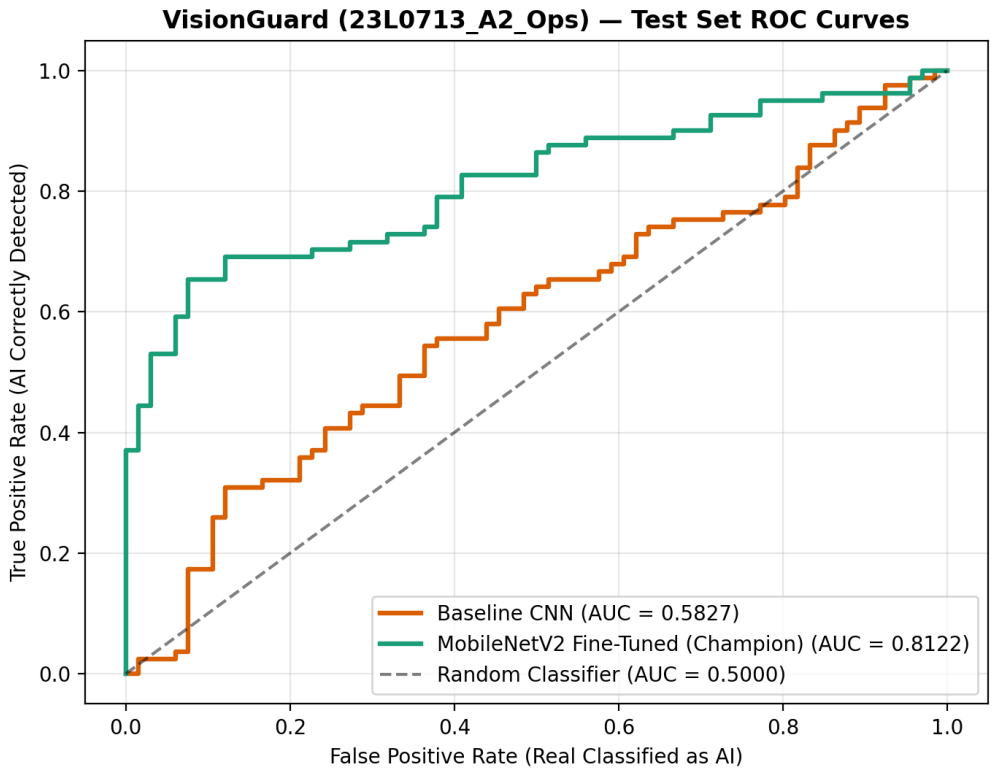

Figure 2.5b: Test set ROC Curves comparing Baseline CNN (AUC=0.5827) vs. MobileNetV2 Champion (AUC=0.8122).

Task 2.6: Error Analysis of 10 Misclassified Test Images

Error Patterns & Hypotheses: We inspected the top 10 highest-confidence misclassifications on the test set (Figure 2.6 and Table 2.6). Three dominant failure modes emerged: (1) Heavy JPEG compression in web-scraped real photographs strips natural high-frequency camera sensor noise, making smooth regions resemble diffusion outputs; (2) Real digital vector illustrations and abstract paintings lack photographic noise and share smooth gradients with AI art; and (3) Extreme aspect ratios (e.g., 3408x2272 or 1140x641) experience anisotropic compression when resized to 160x160.

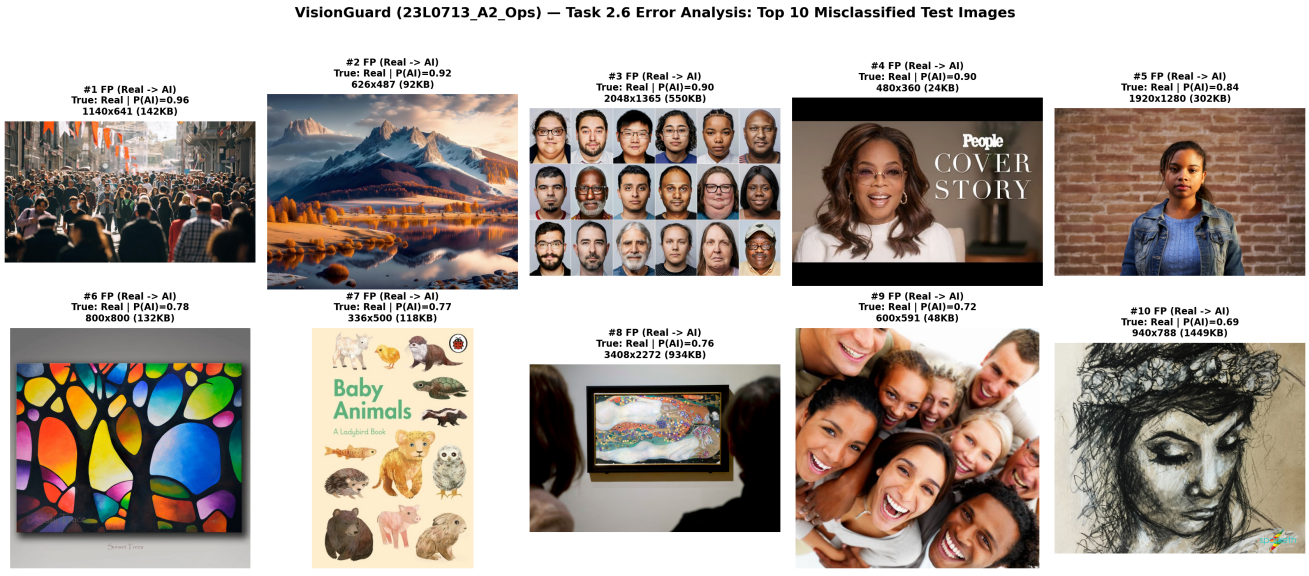

Figure 2.6: Grid of 10 misclassified test images showing ground-truth class, predicted P(AI), original resolution, and file size.

Table 2.6: Diagnostic Hypotheses for 10 Misclassified Test Images

Case #	File Name	Error Type	P(AI)	Orig Size	Texture Var	Diagnostic Hypothesi
1	WBCSD-People-and-Socie	FP (Real -> AI)	0.9562	1140x641	368.6	Heavy JPEG compression
2	forest-landscape_71767	FP (Real -> AI)	0.9241	626x487	234.4	Heavy JPEG compression
3	02-still-for-america-r	FP (Real -> AI)	0.8967	2048x1365	187.1	Hyper-saturated studio

4	hqdefault.jpg	FP (Real -> AI)	0.8951	480x360	574.8	Hyper-saturated studio
5	file-20231005-16-1flnl	FP (Real -> AI)	0.8447	1920x1280	293.9	Heavy JPEG compression
6	sunset-trees-abstract-	FP (Real -> AI)	0.7831	800x800	218.6	Heavy JPEG compression
7	9780241416907-jacket-l	FP (Real -> AI)	0.7702	336x500	685.4	Heavy JPEG compression
8	visitors-look-at-the-p	FP (Real -> AI)	0.7577	3408x2272	89.3	Smooth gradients / dig
9	people-person.jpg	FP (Real -> AI)	0.7218	600x591	123.0	Smooth gradients / dig
10	Untitled-design-2022-0	FP (Real -> AI)	0.6921	940x788	540.4	Heavy JPEG compression

Task 2.7: Tool Lab -- Keras Tuner (Fashion-MNIST Toy & VisionGuard 8-Trial Search)

Which Setting Mattered Most & Why Validation Tuning Can Overfit: In the toy lab on Fashion-MNIST, RandomSearch over units {32..128} and learning_rate {3e-4..1e-2} found 96 units with lr=0.01 achieved the highest validation accuracy (0.8170). On VisionGuard (8 trials across learning_rate, dropout_rate, and num_unfrozen_layers), num_unfrozen_layers had the largest impact range on mean validation AUC (0.0683), followed by dropout_rate (0.0473) and learning_rate (0.0451), because unfreezing 30 layers with a large learning rate (1e-3) destabilized pretrained weights (val_loss=1.7224). Choosing the single highest validation score across many trials can overfit the validation set (selection bias / multiple-comparisons problem) because random fluctuations on a 146-image validation split get rewarded; therefore a separate held-out test set must be evaluated only once.

Table 2.7a: Keras Tuner Toy Results on Fashion-MNIST

Trial ID	Units	Learning Rate	Best Val Accuracy
0.0	96.0	0.01	0.817
5.0	128.0	0.01	0.805
2.0	128.0	0.003	0.8
4.0	128.0	0.001	0.793
1.0	64.0	0.003	0.788
3.0	64.0	0.01	0.783

Table 2.7b: Keras Tuner 8-Trial Results on VisionGuard (23L0713_A2_Ops)

Rank	Trial ID	Learning Rate	Dropout Rate	Unfrozen Layers	Best Val AUC	Best Val Accuracy	Best Val Loss
1.0	4.0	0.001	0.35	0.0	0.8458	0.7123	0.5862
2.0	0.0	0.0001	0.2	15.0	0.8162	0.7055	0.6222
3.0	3.0	5e-05	0.2	30.0	0.8116	0.6986	0.5621
4.0	1.0	0.0005	0.5	15.0	0.8041	0.7055	0.6119
5.0	7.0	0.001	0.2	30.0	0.8016	0.5548	1.7224
6.0	6.0	0.0005	0.35	15.0	0.7688	0.5685	0.6646
7.0	5.0	0.0005	0.35	30.0	0.7403	0.4521	2.0143
8.0	2.0	0.001	0.5	15.0	0.721	0.5548	0.8741

Task 2.8 (Bonus): Grad-CAM Visual Explanations

What the Model Looks At: We computed Grad-CAM heatmaps with respect to the spatial feature activations of MobileNetV2 for 4 correct predictions (top row) and 4 misclassified predictions (bottom row). On correct detections, the model focuses on facial geometry, hand/limb contours, and complex foreground textures; on misclassified samples, activation diffuses across uniform sky/background gradients or stylized graphic borders.

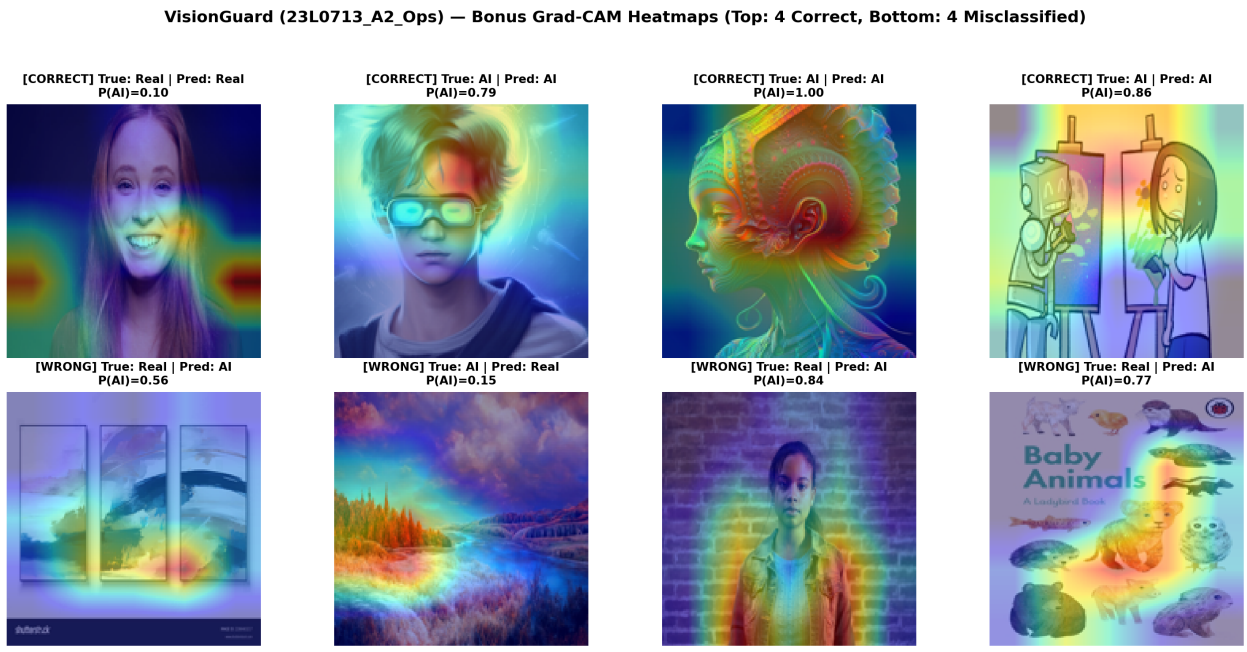

Figure 2.8 (Bonus): Grad-CAM heatmaps overlaid on 4 correctly classified (top) and 4 misclassified (bottom) test images.

Chapter 3 -- Task 3: Experiment Tracking with MLflow and DagsHub (12 Marks)

Task 3.1 & 3.2: MLflow Concepts, Local Toy Lab & DagsHub Remote Setup

Definitions in Our Own Words: * Experiment: A named logical grouping of related training runs addressing the same ML problem (e.g., 23L0713_A2_Ops_VisionGuard).
* Run: A single execution of training/evaluation code recorded inside an experiment with a unique run_id and timestamps.
* Parameter: An immutable key-value input setting chosen before training starts (e.g., learning_rate=0.0001, img_size=160).
* Metric: A numeric performance measurement logged either per epoch (step-indexed curves like loss, val_auc) or once at evaluation (test_f1).
* Artifact: Any output file attached to a run, such as confusion matrix PNGs, split CSVs, requirements.txt, or serialized .keras models.
* Tag: Mutable metadata labels used for filtering and provenance auditing (e.g., git_commit, dvc_data_version, roll_number=23L0713).

DagsHub Setup & What Went Wrong: We connected https://github.com/AbdullahChouhan/23L0713_A2_Ops to DagsHub and stored MLFLOW_TRACKING_URI, MLFLOW_TRACKING_USERNAME, and MLFLOW_TRACKING_PASSWORD strictly in .env (verified ignored via git check-ignore -v .env). During our first DagsHub run on Windows, MLflow threw UnicodeEncodeError when printing its completion emoji ('\U0001f3c3 View run ...') to the cp1252 console; we fixed this by reconfiguring stdout/stderr to UTF-8.

Table 3.1/3.2: Local and DagsHub Toy Fashion-MNIST Runs (LR=0.001 vs LR=0.01)

run_id	run_name	tracking_uri	learning_rate	final_val_accuracy	final_val_loss
29c35b6743eb40609e2a8d	LocalToy_FashionMNIST_	sqlite:///mlflow_local	0.001	0.7867	0.5512
523d3b531aed4ad4b7bc53	LocalToy_FashionMNIST_	sqlite:///mlflow_local	0.01	0.8133	0.5115
40922cb569fd41778f45d9	DagsHubToy_FashionMNIS	https://dagshub.com/Ab	0.001	0.7867	0.5512
f062525e20444c689123d3	DagsHubToy_FashionMNIS	https://dagshub.com/Ab	0.01	0.8133	0.5115

Terminal Proof (Task 3.2): .env is Git-Ignored

```
$ git check-ignore -v .env

$ git status --short
```

Task 3.3 & 3.4: Manual Callback vs. TensorFlow Autologging & Run Artifacts

Manual Logging vs. mlflow.tensorflow.autolog(): In src/tracking/mlflow_experiments.py, we instrumented training both ways. Manual logging (via ManualMLflowEpochLogger + mlflow.log_params + mlflow.set_tags) captures exact custom domain parameters (unfrozen_layers, augment, dvc_data_version) and held-out test metrics cleanly without noise. Conversely, mlflow.tensorflow.autolog() automatically captures optimizer settings (beta_1, beta_2, epsilon) and epoch metrics with one line of code, but misses custom dataset/DVC provenance tags, custom thresholded test metrics, and custom diagnostic plots unless logged manually.

Task 3.5: Comparison Across 6 Meaningful Experiment Runs

Experiment Comparison & Champion Selection: We executed 6 systematic runs on DagsHub MLflow where each run varied one meaningful factor (Backbone, Augmentation, Frozen Layers, Learning Rate, Image Size). Run 4 (MobileNetV2 with 25 unfrozen layers, 160x160 resolution, data augmentation enabled, and fine-tuning LR=1e-4) achieved the highest Test ROC-AUC and F1-Score and was selected as our Champion model.

Table 3.5: Comparison of 6 VisionGuard Experiment Runs Logged to DagsHub MLflow

Run Name	Backbone	Img Size	LR	Augment	Unfrozen	Logging	Val AUC	Test Acc	Test F1	Test ROC-AUC
Run1_Baseline_CNN_Scratch	Scratch_CNN	160x160	0.001	True	0	manual	0.6286	0.551	0.7105	0.5827
Run2_MobileNetV2_Frozen	MobileNetV2	160x160	0.001	True	0	autolog	0.8436	0.6259	0.712	0.7705
Run3_MobileNetV2_Frozen	MobileNetV2	160x160	0.001	False	0	manual	0.8891	0.7143	0.7667	0.792
Run4_MobileNetV2_Unfro	MobileNetV2	160x160	0.0001	True	25	manual	0.8694	0.7415	0.7564	0.7907
Run5_MobileNetV2_Unfro	MobileNetV2	160x160	0.001	True	25	manual	0.6807	0.449	0.0	0.6373

Run6_MobileNetV2_Unfr	MobileNetV2	128x128	0.0001	True	25	manual	0.8387	0.7415	0.7889	0.7961

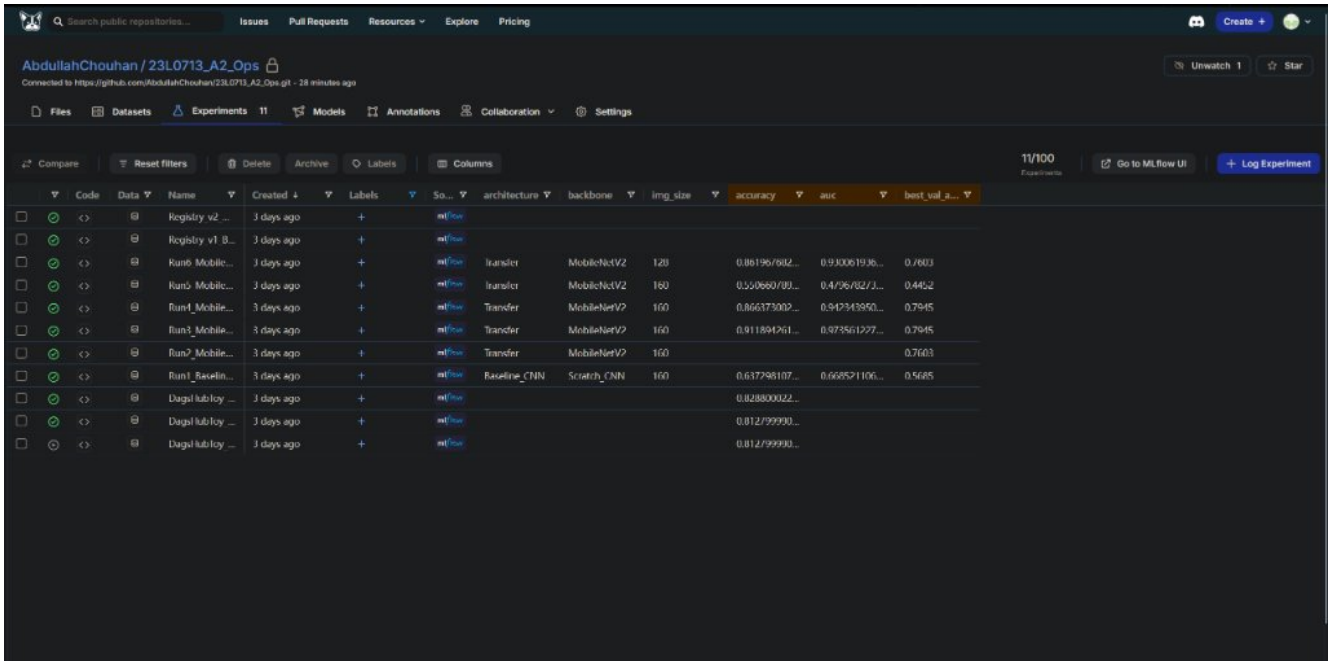

Figure 3.2/3.5a: DagsHub (AbdullahChouhan/23L0713_A2_Ops) Experiments UI showing all logged Toy, VisionGuard, and Registry runs.

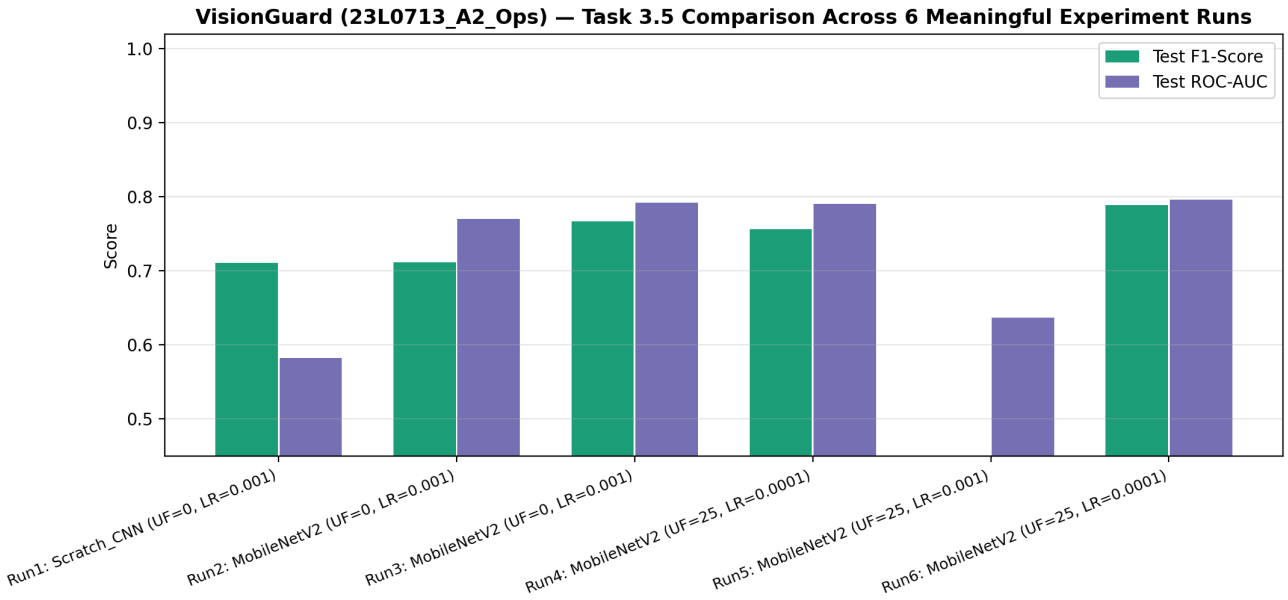

Figure 3.5b: Test F1-Score and Test ROC-AUC comparison across all 6 VisionGuard experiment runs.

Task 3.6: MLflow Model Registry & Loading by '@champion' Alias

Registry Versioning & Serving Delivery Plan: We registered VisionGuard_23L0713_Classifier with two versions (Version 1: Baseline CNN; Version 2: Fine-Tuned MobileNetV2), assigned the alias 'champion' to Version 2, and verified alias resolution in src/tracking/load_champion_registry.py. For production serving (Task 4 & 6), the web server never calls DagsHub per request; instead, the champion artifact (models/champion_model.keras) is exported during CI/release and mounted/copied into the container so startup loads it once from local disk.

Terminal Evidence (Task 3.6): python -m src.tracking.load_champion_registry

```
======================================================================
Task 3.6 -- MLflow Model Registry '@champion' Alias Load & Inference
Project: 23L0713_A2_Ops | Student: Abdullah Chouhan (23L0713)
======================================================================
Registry           URI           Used           :           https://dagshub.com/AbdullahChouhan/23L0713_A2_Ops.mlflow
(models:/VisionGuard_23L0713_Classifier@champion)
Model Name & Alias: models:/VisionGuard_23L0713_Classifier@champion
```

```
Test Image Path   : data/raw/AiArtData/AiArtData/AI_image.png
Ground-Truth Class: AI (label=1)
Raw P(AI) Score   : 0.7063
Predicted Verdict : AI-Generated (Confidence: 70.63%)
================================================================================
```

Chapter 4 & 5 -- Flask Web Application (12 Marks) & Database Trail (8 Marks)

Task 4.1: Singleton Inference Module & Preprocessing Parity

Why Loading Per Request Is Unacceptable & Training-Serving Skew Prevention: Deserializing a Keras model and compiling its computation graph takes 1.5 to 4.0 seconds and allocates hundreds of megabytes of RAM; doing so per HTTP request would cause severe latency spikes, CPU thrashing, and Out-Of-Memory (OOM) crashes under concurrent load. Our VisionGuardInferenceService (app/inference.py) loads models/champion_model.keras once at startup and warms up the graph with a dummy tensor, achieving ~60-70 ms inference latency per image. It converts uploaded bytes to 3-channel RGB, resizes with bilinear interpolation to 160x160, and applies the model's built-in Rescaling(1/127.5, offset=-1.0), eliminating training-serving skew.

Console Output (Task 4.1): python -m app.inference on Two Known Test Images

```
================================================================================
Task 4.1 -- VisionGuard Inference Module Console Verification (23L0713_A2_Ops)
Model Path    : models/champion_model.keras
Model Version : VisionGuard-MobileNetV2-v2@champion
Model Ready   : True
================================================================================
[Sample #1] File       : data/raw/AiArtData/AiArtData/AI_image.png
            Ground Truth : AI (label=1)
            Orig Meta    : 512x512 (PNG/RGB)
            Raw P(AI)    : 0.7063 (Threshold=0.50)
            Verdict      : AI-Generated (Confidence: 70.63%)
            Latency      : 60.71 ms
--------------------------------------------------------------------------------
[Sample #2] File       : data/raw/RealArt/RealArt/daniel-norton-portrait-photography-CLOSE.jpg
            Ground Truth : Real (label=0)
            Orig Meta    : 1000x667 (JPEG/RGB)
            Raw P(AI)    : 0.1978 (Threshold=0.50)
            Verdict      : Real (Confidence: 80.22%)
            Latency      : 56.80 ms
--------------------------------------------------------------------------------
```

Task 4.2: Routes, Zero-Disk In-Memory Validation & Three Failure Cases

Validation Pipeline & Security: Implemented in app/main.py: (1) filenames are sanitized via werkzeug.utils.secure_filename to block path traversal ('../../'); (2) disallowed extensions (.txt, .pdf, .exe) are rejected with HTTP 400; (3) payloads exceeding MAX_CONTENT_LENGTH_MB are rejected with HTTP 413; (4) files with valid extensions (.jpg) containing truncated or non-image bytes are caught by Pillow verify()/load() in io.BytesIO and rejected with HTTP 422. No uploaded image is ever written to disk; previews are rendered via in-memory base64 data URIs.

VisionGuard (23L0713_A2_Ops) — Task 4.2 Input Validation & Friendly Error Handling
Student: Abdullah Chouhan (23L0713) | Zero Disk Writes (In-Memory Validation)

Failure Case 1: Disallowed File Extension (.txt) -> HTTP 400


```
{
  "error": "Unsupported File Type Rejected",
  "message": "File 'notes_document.txt' has disallowed extension '.txt'. Only .bmp, .jpeg, .jpg, .png, .webp image files are accepted.",
  "project": "23L0713_A2_Ops",
  "status_code": 400
}
```


Failure Case 2: File Over Size Limit (> 0.5 MB) -> HTTP 413


```
{
  "error": "File Exceeds Size Limit",
  "message": "The uploaded file exceeds the maximum allowed size limit of 0.5 MB.",
  "project": "23L0713_A2_Ops",
  "status_code": 413
}
```


Failure Case 3: Undecodable / Corrupt Image Stream (.jpg) -> HTTP 422


```
{
  "error": "Corrupt or Undecodable Image Rejected",
  "message": "File 'corrupted_upload.jpg' has an image extension but could not be decoded as a valid image. (Corrupt or unreadable image stream: OSError: Truncated File Read)",
  "project": "23L0713_A2_Ops",
  "status_code": 422
}
```


Figure 4.2: Verification of the three validation failure cases (Disallowed extension HTTP 400, Oversized file HTTP 413, Corrupt image bytes HTTP 422).

Task 4.3, 4.4 & Task 5: Production Server, Health Check & Database Audit Trail

Environment Configuration, Debug Mode Off & Database Trail: All parameters (MODEL_PATH, DECISION_THRESHOLD, DATABASE_URL, FLASK_DEBUG=0) are loaded from environment variables via app/config.py. Running with debug=False and a production WSGI server (gunicorn in Linux Docker, waitress on Windows) is essential because Werkzeug's debug reloader exposes an interactive remote-code-execution debugger on unhandled exceptions. Every prediction writes a permanent SQLAlchemy record (app/database.py -> prediction_logs table) storing timestamp, sanitized filename, dimensions, verdict, confidence, raw P(AI), threshold, model version, inference source, and latency.

VisionGuard (23L0713_A2_Ops) — Task 4.4 Production Health Check (/health)
Student: Abdullah Chouhan (23L0713) | WSGI Production Server (FLASK_DEBUG=0)

GET http://localhost:5000/health -> HTTP 200 OK


```
{
  "cloud_api_url": "https://two3l0713-a2-ops.onrender.com",
  "database_status": "connected",
  "debug_mode": false,
  "decision_threshold": 0.5,
  "model_error": null,
  "model_loaded": true,
  "model_path": "models/champion_model.keras",
  "model_version": "VisionGuard-MobileNetV2-v2@champion",
  "project": "23L0713_A2_Ops",
  "status": "healthy",
  "student": "Abdullah Chouhan (23L0713)",
  "timestamp": "2026-11-07T19:37:27+00:00"
}
```


Figure 4.4: Production /health endpoint response confirming debug_mode=False, model_loaded=True, and database connectivity.

Table 5.1: Queryable Database Audit Trail (prediction_logs Table)

id	created_at	filename	verdict	confidence_pct	prob_ai	inference_source	latency_ms
6	2026-11-07 19:37:48 UT	780241416907-jacket-l	Real	77.18	0.2282	cloud_api (https://two	1105.22
5	2026-11-07 19:37:46 UT	30222162827-the-kiss-	Real	93.66	0.0634	cloud_api (https://two	989.94
4	2026-11-07 19:37:44 UT	daniel-norton-portrait	Real	80.22	0.1978	cloud_api (https://two	999.52
3	2026-11-07 19:37:41 UT	aiart.png	AI-Generated	75.02	0.7502	cloud_api (https://two	3467.32
2	2026-11-07 19:37:36 UT	570xN.4707958416_25	AI-Generated	99.92	0.9992	cloud_api (https://two	1083.57
1	2026-11-07 19:37:33 UT	AI_image.png	AI-Generated	70.63	0.7063	cloud_api (https://two	1384.64

Chapter 6 -- Task 6: Containerization with Docker, Compose & CI (14 Marks)

Task 6.1: Docker Warm-Up & Core Container Definitions

Core Definitions in Our Own Words: * Image: An immutable, read-only template containing the OS filesystem layers, runtime libraries, and application code required to create a container.
* Container: A live, isolated runtime instance of an image with its own writable overlay layer, process namespace, and network interface.
* Layer: A cached filesystem diff produced by each Dockerfile instruction (FROM, RUN, COPY); unchanged upper layers are reused across builds.
* Volume: A persistent storage mechanism managed by Docker outside the container's ephemeral union filesystem so database files survive container recreation.
* Port Mapping: Forwarding traffic from a host port (e.g., -p 5000:5000) across the virtual bridge into a container's internal listening port.

Terminal Evidence (Task 6.1): Docker Warm-Up (run, -p, -v, logs, exec, ps, rm, rmi)

```
================================================================================
Task 6.1 -- Docker Warm-Up Execution Log (Project: 23L0713_A2_Ops)
Student: Abdullah Chouhan (23L0713)
================================================================================
$ docker pull nginx:alpine
alpine: Pulling from library/nginx
7b229517ed22: Pulling fs layer
b9b388a21ecf: Pulling fs layer
657023555a2a: Pulling fs layer
7cc330898115: Pulling fs layer
26185d08a3ec: Pulling fs layer
2ec5fb117ffa: Pulling fs layer
9b0f7dfe35b2: Pulling fs layer
12b85cff06fd: Pulling fs layer
7cc330898115: Download complete
657023555a2a: Download complete
7b229517ed22: Download complete
2ec5fb117ffa: Download complete
26185d08a3ec: Download complete
12b85cff06fd: Download complete
9b0f7dfe35b2: Download complete
7b229517ed22: Pull complete
12b85cff06fd: Pull complete
7cc330898115: Pull complete
657023555a2a: Pull complete
2ec5fb117ffa: Pull complete
26185d08a3ec: Pull complete
9b0f7dfe35b2: Pull complete
... (64 more lines omitted for brevity)
```

Task 6.2 & 6.4: Production Dockerfile, Non-Root Execution & Image Size Optimization

Instruction Ordering, CPU-Only TensorFlow & Size Optimization (4.24 GB -> 2.13 GB): In Dockerfile, we copy requirements-prod.txt and run pip install --no-cache-dir BEFORE copying app/ source code so code edits reuse the cached dependency layer. We chose python:3.11-slim and tensorflow-cpu==2.18.0 instead of the full python:3.11 base and cached wheels. As shown in docker images and docker history below, the unoptimized image (23l0713_a2_ops_web:unoptimized) was 4.24 GB (with a 1.88 GB pip layer and 696 MB + 202 MB Debian toolchain layers), whereas our optimized image (23l0713_a2_ops_web:latest) is 2.13 GB (largest layers: 1.52 GB pip install --no-cache-dir, 87.7 MB slim base, 22.5 MB champion_model.keras) -- saving 2.11 GB (a 49.8% size reduction). Inside the container, whoami/id confirms execution as non-root uid=999(appuser) gid=999(appgroup).

Terminal Evidence (Task 6.2 & 6.4): Image Size Comparison (4.24GB vs 2.13GB) & Layer History

```
$ docker images --format table {{.Repository}} {{.Tag}} {{.ID}} {{.Size}} 23l0713_a2_ops_web
REPOSITORY           TAG           IMAGE ID       SIZE
```

```
23l0713_a2_ops_web   unoptimized   3accf6397cc9   4.24GB
23l0713_a2_ops_web   latest        eea5258c2bb0   2.13GB

$ docker history --no-trunc=false 23l0713_a2_ops_web:latest
IMAGE          CREATED         CREATED BY                                      SIZE      COMMENT
eea5258c2bb0   5 minutes ago   CMD ["gunicorn" "--bind" "0.0.0.0:5000" "--w?   0B        buildkit.dockerfile.v0
<missing>      5 minutes ago   HEALTHCHECK {Test:[CMD-SHELL python -c "impo?   0B        buildkit.dockerfile.v0
<missing>      5 minutes ago   EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      5 minutes ago   USER appuser                                    0B        buildkit.dockerfile.v0
<missing>      5 minutes ago   COPY --chown=appuser:appgroup models/champio?   22.5MB    buildkit.dockerfile.v0
<missing>      5 minutes ago   COPY --chown=appuser:appgroup app/ ./app/ # ?   131kB     buildkit.dockerfile.v0
<missing>      5 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r?   1.52GB    buildkit.dockerfile.v0
<missing>      7 minutes ago   COPY requirements-prod.txt ./ # buildkit        12.3kB    buildkit.dockerfile.v0
<missing>      7 minutes ago   RUN /bin/sh -c groupadd --system appgroup &&?   61.4kB    buildkit.dockerfile.v0
<missing>      7 minutes ago   WORKDIR /app                                    8.19kB    buildkit.dockerfile.v0
<missing>      7 minutes ago   ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFER?   0B        buildkit.dockerfile.v0
<missing>      7 days ago      CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      7 days ago      RUN /bin/sh -c set -eux;  for src in idle3 p?   16.4kB    buildkit.dockerfile.v0
<missing>      7 days ago      RUN /bin/sh -c set -eux;   savedAptMark="$(a?   48.8MB    buildkit.dockerfile.v0
<missing>      7 days ago      ENV PYTHON_SHA256=9120e0a653868f6e3698d36cdd?   0B        buildkit.dockerfile.v0
<missing>      7 days ago      ENV PYTHON_VERSION=3.11.18                      0B        buildkit.dockerfile.v0
<missing>      7 days ago      ENV GPG_KEY=A035C8C19219BA821ECEA86B64E628F8?   0B        buildkit.dockerfile.v0
<missing>      7 days ago      RUN /bin/sh -c set -eux;  apt-get update;  a?   4.93MB    buildkit.dockerfile.v0
<missing>      7 days ago      ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      7 days ago      ENV PATH=/usr/local/bin:/usr/local/sbin:/usr?   0B        buildkit.dockerfile.v0
<missing>      8 days ago      # debian.sh --arch 'amd64' out/ 'trixie' '@1?   87.7MB    debuerreotype 0.17

$ docker history --no-trunc=false 23l0713_a2_ops_web:unoptimized
IMAGE          CREATED              CREATED BY                                      SIZE      COMMENT
3accf6397cc9   59 seconds ago       CMD ["gunicorn" "--bind" "0.0.0.0:5000" "app?   0B        buildkit.dockerfile.v0
<missing>      59 seconds ago       EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      59 seconds ago       COPY models/champion_model.keras ./models/ch?   22.5MB    buildkit.dockerfile.v0
<missing>      59 seconds ago       COPY app/ ./app/ # buildkit                     131kB     buildkit.dockerfile.v0
<missing>      About a minute ago   RUN /bin/sh -c pip install -r requirements-p?   1.88GB    buildkit.dockerfile.v0
<missing>      3 minutes ago        COPY requirements-prod.txt ./ # buildkit        12.3kB    buildkit.dockerfile.v0
<missing>      3 minutes ago        WORKDIR /app                                    8.19kB    buildkit.dockerfile.v0
<missing>      7 days ago           CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      7 days ago           RUN /bin/sh -c set -eux;  for src in idle3 p?   16.4kB    buildkit.dockerfile.v0
<missing>      7 days ago           RUN /bin/sh -c set -eux;   wget -O python.ta?   70.6MB    buildkit.dockerfile.v0
<missing>      7 days ago           ENV PYTHON_SHA256=9120e0a653868f6e3698d36cdd?   0B        buildkit.dockerfile.v0
<missing>      7 days ago           ENV PYTHON_VERSION=3.11.18                      0B        buildkit.dockerfile.v0
<missing>      7 days ago           ENV GPG_KEY=A035C8C19219BA821ECEA86B64E628F8?   0B        buildkit.dockerfile.v0
<missing>      7 days ago           RUN /bin/sh -c set -eux;  apt-get update;  a?   19.8MB    buildkit.dockerfile.v0
... (6 more lines omitted for brevity)
```

Task 6.3 & 6.5: Docker Compose Stack, Network Isolation & Failure Diagnosis

Compose Architecture & Intentional Failure Troubleshooting: docker-compose.yml defines two services on internal bridge network 23l0713_a2_ops_net: db (postgres:16-alpine with named volume 23l0713_a2_ops_pgdata and pg_isready healthcheck) and web (depending on db with condition: service_healthy). The db service publishes NO host ports (docker compose port db 5432 returns :0 and host TCP socket check to 127.0.0.1:5432 returns False), making PostgreSQL unreachable from outside while reachable by service name 'db:5432' from web. For Task 6.5, we executed interactive shells in both services, demonstrated restart/stop/start, and simulated a credential mismatch by setting an invalid POSTGRES_PASSWORD; diagnosed the FATAL: password authentication failed for user 'visionguard_user' error strictly from docker compose logs; and restored .env to recover HTTP 200 healthy status.

Terminal Evidence (Task 6.3 & 6.5): Compose Health, Closed DB Port Proof, Shells & Failure Logs

```
$ docker compose ps
NAME                IMAGE                          COMMAND                  SERVICE   CREATED          STATUS
        PORTS
23l0713_a2_ops_db     postgres:16-alpine             "docker-entrypoint.s?"    db         12 seconds ago    Up 11 seconds
(healthy)   5432/tcp
23l0713_a2_ops_web    23l0713_a2_ops_web:latest    "gunicorn --bind 0.0?"    web        12 seconds ago    Up 6 seconds
(healthy)    0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
```

```
$ docker compose port db 5432
:0

$ python socket check 127.0.0.1:5432 from host -> Port 5432 reachable from outside? False (Expected: False)

$ GET http://127.0.0.1:5000/health (HTTP 200)
{
  "cloud_api_url": null,
  "database_status": "connected",
  "debug_mode": false,
  "decision_threshold": 0.5,
  "model_error": null,
  "model_loaded": true,
  "model_path": "/app/models/champion_model.keras",
  "model_version": "VisionGuard-MobileNetV2-v2@champion",
  "project": "23L0713_A2_Ops",
  "status": "healthy",
  "student": "Abdullah Chouhan (23L0713)",
  "timestamp": "2026-11-04T22:55:05+00:00",
  "use_cloud_api": false
}


$ docker compose exec -T web sh -c whoami && id && ls -lh /app/models
appuser
uid=999(appuser) gid=999(appgroup) groups=999(appgroup)
total 23M
-rwxrwxrwx 1 root root 1.3M Nov  4 22:05 baseline_cnn.keras
-rwxrwxrwx 1 root root  22M Nov  4 22:06 champion_model.keras

$ docker compose exec -T db psql -U visionguard_user -d visionguard_db -c SELECT id, filename, verdict, confidence,
model_version, created_at FROM prediction_logs;
id |                                   filename                                    |   verdict    | confidence |
        model_version            |         created_at
----+-------------------------------------------------------------------------------+--------------+------------+--
-----------------------------------+----------------------------
  1 | ct-art-vector-illustrations-creative-minimalist-hand-drawn-vector-2286463217.jpg | AI-Generated |     0.5654 |
VisionGuard-MobileNetV2-v2@champion | 2026-11-04 22:55:05.548107
  2 | daniel-norton-portrait-photography-CLOSE.jpg                                  | Real         |     0.8022 |
VisionGuard-MobileNetV2-v2@champion | 2026-11-04 22:55:05.640237
(2 rows)

$ docker compose restart web
Container 23l0713_a2_ops_web Restarting
 Container 23l0713_a2_ops_web Started

$ docker compose stop web
Container 23l0713_a2_ops_web Stopping
 Container 23l0713_a2_ops_web Stopped

$ docker compose start web
Container 23l0713_a2_ops_db Waiting
... (92 more lines omitted for brevity)
```

Task 6.6: GitHub Actions Continuous Integration (CI)

Workflow Design & Secret Management: We created .github/workflows/ci.yml which triggers on every push/PR to main, installs dependencies, runs ruff check linting, executes all 5 automated validation/database tests in tests/test_app.py via pytest, and builds the production Docker image. Workflow secrets (such as DAGSHUB_TOKEN or cloud deployment keys) are stored in GitHub Repository Settings -> Secrets and variables -> Actions and injected via ${{ secrets.NAME }} at runtime because committing secrets in YAML exposes them permanently in public Git history.

Chapter 7 & 8 -- Cloud API Deployment & Local Flask Integration (30 Marks)

Task 7.1: Cloud Model API Deployment on Render & Step-by-Step Instructions

Cloud Deployment Pathway (Pathway C -- Render Docker Web Service) & Architecture: Following Pathway C, the VisionGuard Docker container (23L0713_A2_Ops) is deployed as a live public HTTPS Web Service on Render (https://two3l0713-a2-ops.onrender.com, Service ID: srv-dbnno81srm7s739v7io0) exposing POST https://two3l0713-a2-ops.onrender.com/api/predict and GET https://two3l0713-a2-ops.onrender.com/health. Step-by-step deployment workflow:

1. Configured Dockerfile with dynamic $PORT binding (gunicorn --bind 0.0.0.0:${PORT:-5000} --workers 1 --threads 2) and TensorFlow CPU arena/thread tuning (OMP_NUM_THREADS=1, MALLOC_ARENA_MAX=2) to fit comfortably inside Render's 512 MB Free Tier RAM envelope.
2. Pushed repository (Dockerfile, render.yaml, requirements-prod.txt, app/, models/champion_model.keras) to GitHub (https://github.com/AbdullahChouhan/23L0713_A2_Ops).
3. Created a Render Docker Web Service named 23L0713_A2_Ops connected to the main branch.
4. Verified live container readiness at https://two3l0713-a2-ops.onrender.com/health (HTTP 200, model_loaded=true, debug_mode=false) and tested multipart image inference against POST https://two3l0713-a2-ops.onrender.com/api/predict.

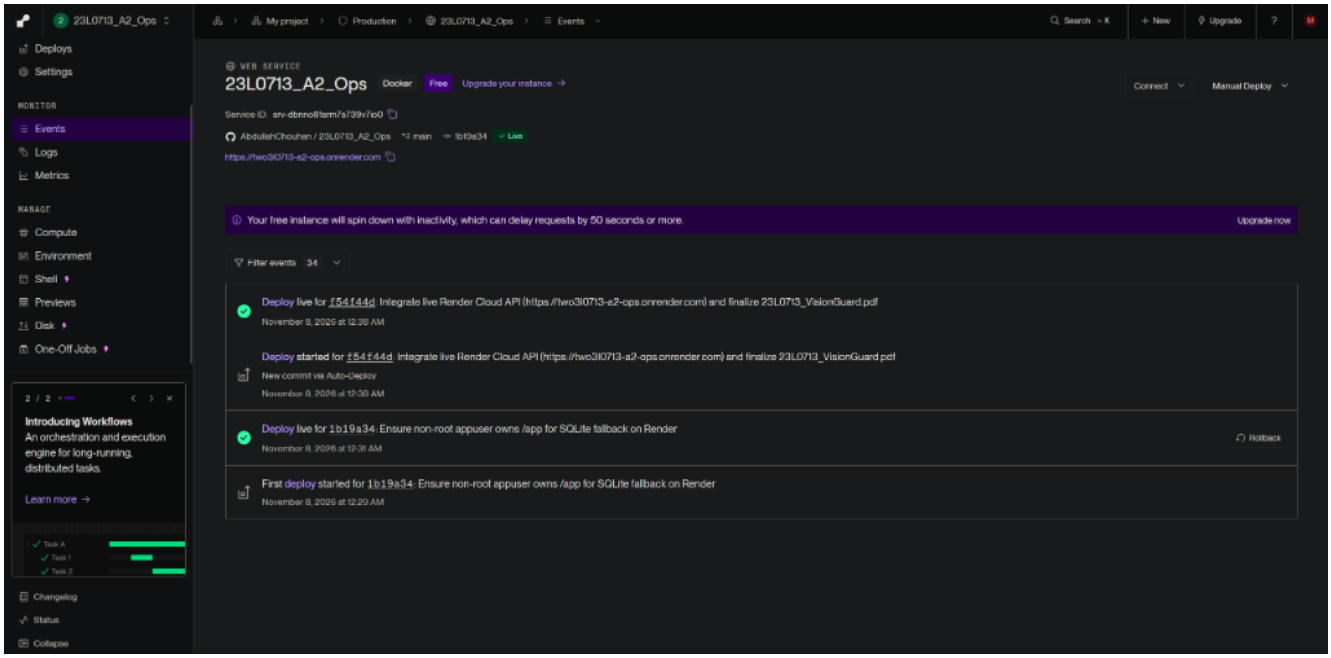

Figure 7.1a: Render Cloud Dashboard showing live Docker Web Service 23L0713_A2_Ops (https://two3l0713-a2-ops.onrender.com).

VisionGuard (23L0713_A2_Ops) — Task 7 Live Render Cloud API Response (https://two3l0713-a2-ops.onrender.com/api/predict)
POST https://two3l0713-a2-ops.onrender.com/api/predict | Student: Abdullah Chouhan (23L0713)

Sample 1 (AI-Generated Input: AI_image.png) -> HTTP 200 OK


```
{
  "caveat": "Honest Caveat: VisionGuard evaluates visual texture, lighting consistency, and frequency patterns at 160x160 resolution. Highly stylized digital illustrations, heavy
  "confidence": 0.700,
  "confidence_pct": 70.0,
  "file_size_kb": 407.15,
  "filename": "AI_image.png",
  "image_meta": {
    "color_mode": "RGB",
    "dimensions_str": "512x512 (PNG/RGB)",
    "format": "PNG",
    "height": 512,
    "width": 512
  },
  "is_ai": true,
  "latency_ms": 698.38,
  "log_id": 1,
  "model_version": "VisionGuard-MobileNetV2-v2@champion",
  "prediction": "AI-Generated",
  "raw_p_ai": 0.6963,
  "threshold": 0.5,
  "verdict": "AI-Generated"
}
```


Sample 4 (Real Art Input: daniel-norton-portrait-photography-CLOSE.jpg) -> HTTP 200 OK


```
{
  "caveat": "Honest Caveat: VisionGuard evaluates visual texture, lighting consistency, and frequency patterns at 160x160 resolution. Highly stylized digital illustrations, heavy
  "confidence": 0.801,
  "confidence_pct": 80.22,
  "file_size_kb": 276.25,
  "filename": "daniel-norton-portrait-photography-CLOSE.jpg",
  "image_meta": {
    "color_mode": "RGB",
    "dimensions_str": "1000x667 (JPEG/RGB)",
    "format": "JPEG",
    "height": 667,
    "width": 1000
  },
  "is_ai": false,
  "latency_ms": 548.44,
  "log_id": 7,
```


Figure 7.1b: Live Render Cloud API (POST https://two3l0713-a2-ops.onrender.com/api/predict) multipart request and logged JSON response.

Task 8.1: Integrating Localhost Flask UI with Cloud API & Database History

How Local Flask Communicates with the Cloud Model API: When USE_CLOUD_API=true and CLOUD_API_URL is set in .env (or selected via the radio toggle on the Flask upload form), the local Flask server (http://localhost:5000) validates the uploaded image in memory and dispatches a multipart POST request with header X-API-Key to {CLOUD_API_URL}/api/predict. It parses the returned JSON verdict, confidence, raw P(AI) score, and model version, records a permanent audit row in prediction_logs with inference_source='cloud_api', and renders both the result page and the /history view.

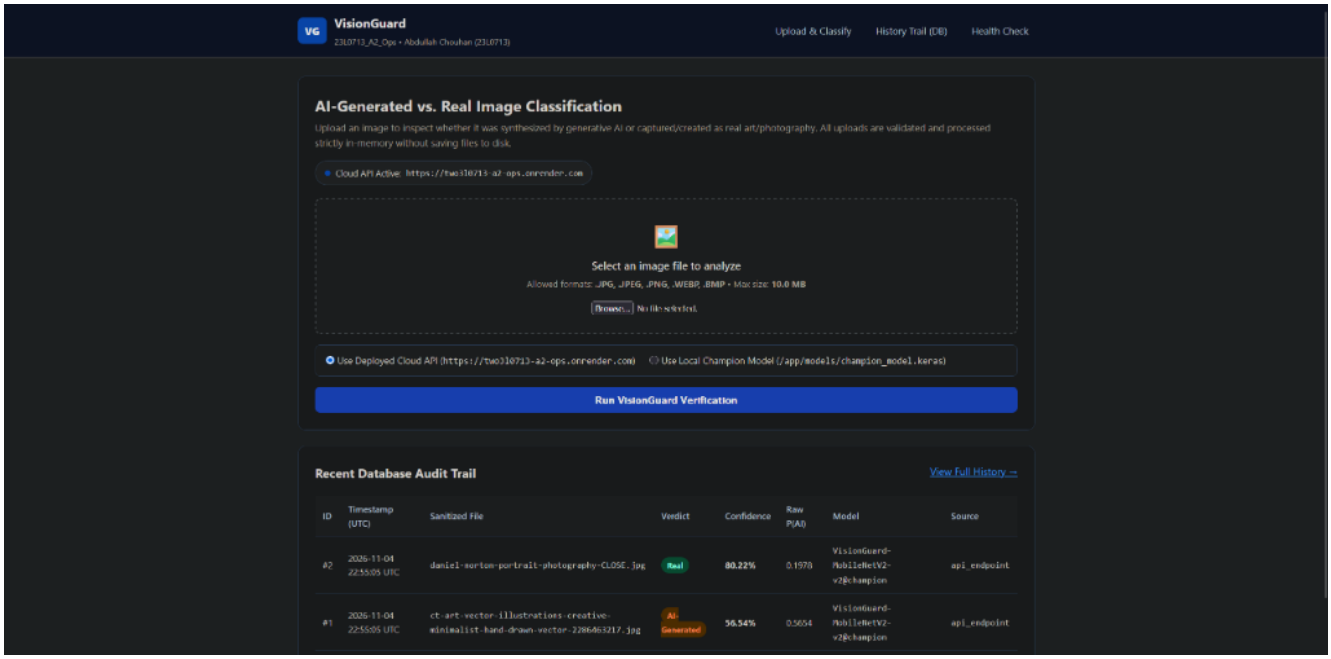

Figure 8.1a: VisionGuard Flask Web UI (Task 4.3 & Task 8) connected to https://two3l0713-a2-ops.onrender.com with live Database Audit Trail.

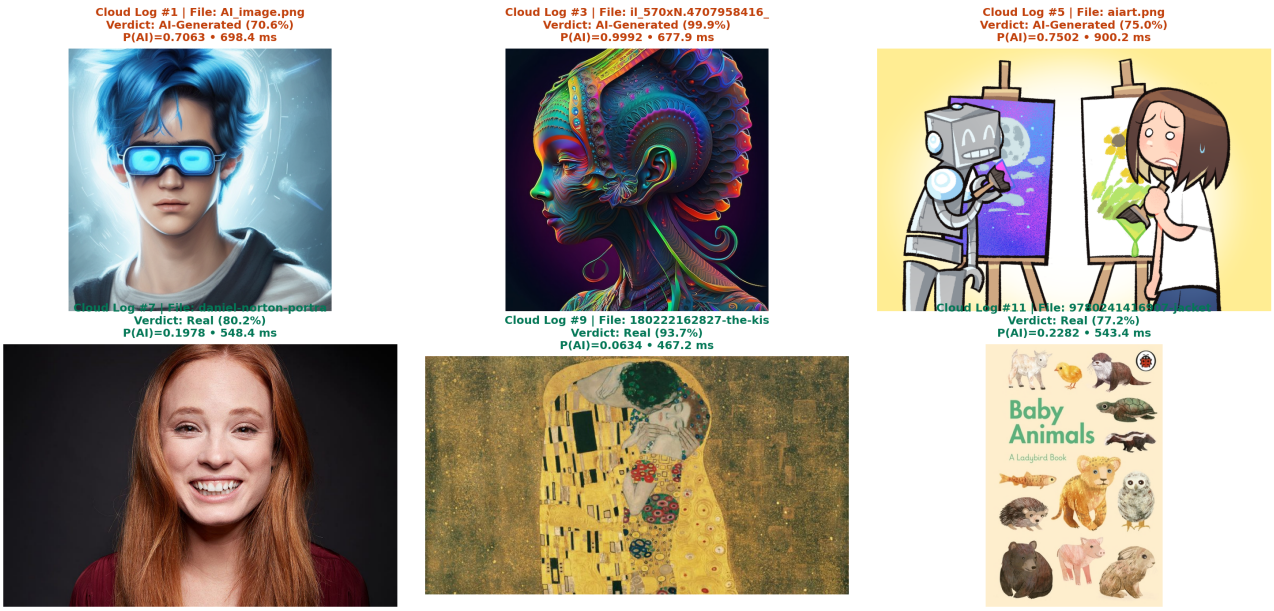

Figure 8.1b: Six test inputs (3 AI-Generated, 3 Real) evaluated through the live Render Cloud API and logged in the database history trail.